

Cairo University

Faculty of Computers and Artificial Intelligence

CS251

**Introduction to
Software
Engineering**

Masroofy

Software Design

Specifications Version 1.0

Name	ID	E-mail	section
Mohamed Yasser	20240541	20240541@stud.fci-cu.edu.eg	s9
Omar abdelaziz	20240375	20240375@stud.fci-cu.edu.eg	s9
Mostafa ayman	20240590	20240590@stud.fci-cu.edu.eg	s29
Mohamed yasser hamed	20240540	20240540@stud.fci-cu.edu.eg	s16

Contents

Instructions [To be removed]	3
Team	3
Document Purpose and Audience	3
System Models	3
I. Architecture Diagram	3
II. Class Diagram(s)	4
III. Class Descriptions	5
IV. Sequence diagrams	6
Class - Sequence Usage Table	7
V. State Diagram	8
VI. SOLID Principles	8
VII. Design Patterns	8
Tools	8
Ownership Report	8

Team

ID	Name	Email	Mobile
20240375	Omar Abd-Elaziz Ali	20240375@stud.fci-cu.edu.eg	01095824650
20240590	Mostafa Ayman Mahmoud	20240590@stud.fci-cu.edu.eg	01151570528
20240541	Mohamed Yasser Mohamed	20240541@stud.fci-cu.edu.eg	01144601498
20240540	Mohamed Yasser Hamed	20240540@stud.fci-cu.edu.eg	01095824650

Document Purpose and Audience

Purpose: This document specifies the software design for our project to ensure all stakeholders and developers understand the system's functionality and constraints.

Audience: The primary audience includes the Course Instructor (Dr. M. El-Ramly), the TA, and the development team.

Opportunities Report

Government-Backed Professional Training Opportunities in Egypt

The Egyptian government, through its various technological arms, offers several high-impact programs designed to bridge the gap between academic study and the professional labor market. These initiatives focus on digital transformation, cloud computing, and cybersecurity.

1. Mahara-Tech (Information Technology Institute - ITI)

Mahara-Tech serves as the official e-learning arm of the ITI. It is designed to provide scalable, high-quality education to a broad audience.

- **Primary Focus:** Self-paced, online learning across diverse tracks including Web Development, Data Science, and Soft Skills.
- **Value Proposition:** Courses are free and widely recognized by Egyptian employers. They serve as an excellent entry point for those looking to build a foundation before applying for intensive 3-month or 9-month ITI diplomas.
- **Access:** maharatech.gov.eg

2. ITI Specialized Cybersecurity (Regional)

The ITI provides specialized, branch-specific tracks to cater to the growing demand for information security professionals in specific governorates.

- **Program Nature:** Intensive technical training focused on network security, penetration testing, and digital forensics.
- **Regional Focus (Giza):** Specific application forms are often released for regional branches to manage local intake.
- **Access:** [Giza Application Portal](#)

3. NTI "HireReady" & "Career Launch"

The **National Telecommunication Institute (NTI)** offers the "HireReady" and "Career Launch" initiatives, which are highly regarded for their practical, job-oriented curriculum.

- **Duration:** 6-month intensive training programs.
- **Core Tracks:** * **Cybersecurity:** Defending infrastructure and managing risks.
 - **DevOps:** Bridging the gap between development and operations for faster delivery.
 - **Telecom Networks:** Advanced training in mobile and fiber-optic communication systems.
- **Outcome:** These programs are specifically engineered to make graduates "market-ready" for immediate employment in large enterprises.
- **Access:** [NTI HireReady](#)

Manara AWS Training

This is a specialized collaboration focusing on **Amazon Web Services (AWS)** to bolster Egypt's presence in the global cloud computing market.

- **Primary Focus:** Cloud architecture, security, and infrastructure management using the AWS ecosystem.
 - **Target Audience:** Software engineers and network professionals looking to migrate or build natively in the cloud.
 - **Strategic Importance:** As more companies move to the cloud, AWS certification and training have become top-tier requirements for high-paying roles in tech.
 - **Access:** [Manara](#)
-

Different Jobs for software engineers

1. Front-end Engineer(the face of website)

He is the bridge between design and technology You take a static design (usually from a tool like Figma) and turn it into a living, breathing website using code.

It requires to be learned:

Languages: HTML(for structure),CSS(for design),Javascript(for dynamic page).

Libraries: Tailwind, Boatstrap.(to make designing page faster)

Frameworks: React or Angular or Vue Js.

How to Freelance as a Frontend Engineer

- Build strong portfolio
- Offer "Figma to Code" Services

2. Back-end Engineer(the hidden part)

If the frontend is the "face" of an application, the Backend is the engine, the brain, and the memory. It's where the data lives, where security is enforced, and where complex logic happens out of sight.

It requires to be learned:

Languages: C# (Strong for enterprise/management systems)

Python (Great for data and speed)

JavaScript (Node.js) (To use JS on the server)

Databases: * SQL (Relational: PostgreSQL, MySQL, SQL Server)

NoSQL (Non-relational: MongoDB)

Frameworks:

ASP.NET Core (C#)

Django (Python)

Express.js (Node.js)

How to Freelance as a Backend Engineer

-Build strong portfolio

-Offer API Services

4. DevOps Engineer (The Systems Architect)

DevOps is the "bridge" between development and operations. If a developer builds the car, you build the **factory** and the **roads**. You make sure the website never crashes and updates automatically.

It requires to be learned:

Cloud Platforms: AWS, Azure, or Google Cloud.

-Automation (CI/CD): Using tools like **GitHub**

Actions to automatically test and upload code.

-Containers: Docker and **Kubernetes** to package software so it runs perfectly on any server.

How to Freelance as a DevOps Engineer

-Offer "Cloud Migration": Help companies move their old data to the cloud safely.

-Cost Optimization: Audit a company's cloud bill and show them how to save money by organizing their servers better.

5. Game Developer (The World Builder)

If the backend is the engine and the frontend is the face, Game Development is the **soul**. You create interactive worlds by combining physics, logic, art, and sound. It is one of the most challenging but rewarding paths in software engineering.

It requires to be learned:

-Game Engines: Unity (uses C# and is great for mobile/VR) or **Unreal Engine** (uses C++ and is the king of high-end graphics).

Mathematics: Strong knowledge of **Vectors, Physics, and Trigonometry** to handle how characters move and collide.

-Scripting: C# or C++ to write the rules of the game (e.g., "if the player hits a spike, they lose a life").

-Asset Creation: Basics of **Blender** (for 3D models) or **Photoshop** (for 2D sprites) so you can put visuals into your engine.

How to Freelance as a Game Developer

Build a "Playable" Portfolio

In game dev, people want to play your work, not just read your code.

-The Move: Participate in **Game Jams** (like on Itch.io) to create small games in 48 hours. Host these games on your website so clients can play them instantly in their browser.

Offer "Niche" Programming Services

Many indie teams have artists but lack "hardcore" programmers for specific systems.

-The Service: Instead of building a whole game, offer to build **specific systems**, like a "Custom Inventory System," "Advanced AI Enemy Behavior," or

"Multiplayer Networking."

Asset Store & Marketplace Passive Income

-The Service: You can write high-quality code templates or tools and sell them on the **Unity Asset Store** or **Unreal Marketplace**. If you build a great "Character Controller," thousands of other developers might buy it for \$20–\$50, giving you income while you sleep.

1. Frontend Development Intern (React.js)

Company: Icura **Location:** Cairo, Egypt

This role focuses on building user-facing web applications using modern JavaScript frameworks.

-Key Technologies: React.js, JavaScript (ES6+), HTML5, and CSS3.

-Basic knowledge of Git version control system and familiarity with agile methodologies.

-Core Responsibilities :

-Assist in the development and implementation of user interface components using React.js.

-Collaborate with the design team to translate UI/UX designs into functional web applications.

Participate in code reviews and contribute to team best practices for coding standards.

-Test, debug, and optimize applications for maximum speed and scalability.

-Support the QA team in troubleshooting issues and performing user acceptance testing. o Access:

<https://app.recruitera.ai/job/frontend-development>

[-intern--react-js--icura-cairo-egypt-3900?ref=DLJ](#)

[24Vsf&igbTracker=902141851](#)

2. Backend Intern

Source: <https://www.linkedin.com/jobs/view/4399634843/>

This position is centered on server-side logic, database management, and ensuring high performance and responsiveness to requests from the frontend.

Focus Areas: API development, server-side languages (Node.js), and database schema design.

Key Responsibilities:

- Develop and maintain server-side applications using Node.js.
- Implement RESTful APIs and microservices architecture to facilitate communication between front-end systems and databases.
- Collaborate with front-end engineers to integrate user-facing elements with server-side logic.
- Ensure code quality and efficiency by performing code reviews and writing unit tests.

Work closely with stakeholders to gather requirements and translate them into technical specifications.

- Monitor application performance and troubleshoot issues, ensuring high availability and reliability.

- Stay updated with industry trends and best practices in backend development within healthcare.

3. Android Intern

Source: <https://www.linkedin.com/jobs/view/4402263514/>

A mobile-specific role focused on the development of applications for the Android operating system.

Technical Requirements:

- Languages: Basic to intermediate proficiency in both Java and Kotlin.

- Android Basics: Understanding of the Android Lifecycle, Activities, Fragments, and basic UI layouts (XML/Compose).

- OOP Principles: Strong grasp of Object-Oriented Programming and clean code logic.

- API Basics: Familiarity with how to consume RESTful APIs (using Retrofit or Volley).

- Problem Solving: Strong analytical skills and a "can-do" attitude toward debugging.

Tools: Basic experience with Git/GitHub for version control.

Key Responsibilities:

-Feature Support: Assist the engineering team in developing and testing small features for our POS applications.

-Bug Fixing: Help identify, document, and resolve minor UI/UX and functional bugs.

-Code Learning: Participate in code reviews to learn best practices and industry standards.

-Documentation: Assist in maintaining technical documentation and testing logs.

-Research: Research specific Android or Payment-related libraries to support senior developers.

4. IT Intern

Source: [LinkedIn](#)

Unlike the development roles above, this position focuses on the infrastructure, hardware, and operational support of a company's technology stack.

Focus Areas:

Current student or recent graduate in IT, Computer Science, or related field

-Basic knowledge of Windows/Linux systems and networking

-Eager to learn, proactive, and a team player

-Good communication and problem-solving skills

key Responsibilities:

-Assist in troubleshooting hardware and software issues

-Support network and system maintenance

-Help with user account management and access controls

- Collaborate on IT projects and documentation
- Learn about IT infrastructure, security, and operations

Screenshoots of comments

3. Android Intern

Source: <https://www.linkedin.com/jobs/view/4402263514/>

A mobile-specific role focused on the development of applications for the Android operating system.

Technical Requirements:

- Languages: Basic to intermediate proficiency in both Java and Kotlin.
- Android Basics: Understanding of the Android Lifecycle, Activities, Fragments, and basic UI layouts (XML/Compose).
- OOP Principles: Strong grasp of Object-Oriented Programming and clean code logic.
- API Basics: Familiarity with how to consume RESTful APIs (using Retrofit or Volley).
- Problem Solving: Strong analytical skills and a "can-do" attitude toward debugging.
- Tools: Basic experience with Git/GitHub for version control.

Key Responsibilities:

- Feature Support: Assist the engineering team in developing and testing small features for our POS applications.
- Bug Fixing: Help identify, document, and resolve minor UI/UX and functional bugs.
- Code Learning: Participate in code reviews to learn best practices and industry standards.
- Documentation: Assist in maintaining technical documentation and testing logs.
- Research: Research specific Android or Payment-related libraries to support senior developers.

4. IT Intern

Source: LinkedIn

Unlike the development roles above, this position focuses on the infrastructure, hardware, and operational support of a company's technology stack.

- **Focus Areas:**
 - Current student or recent graduate in IT, Computer Science, or related field
 - Basic knowledge of Windows/Linux systems and networking
 - Eager to learn, proactive, and a team player
 - Good communication and problem-solving skills
- **Key Responsibilities:**
 - Assist in troubleshooting hardware and software issues
 - Support network and system maintenance
 - Help with user account management and access controls
 - Collaborate on IT projects and documentation
 - Learn about IT infrastructure, security, and operations

1. Frontend Development Intern (React.js)

- **Company:** Icura **Location:** Cairo, Egypt

This role focuses on building user-facing web applications using modern JavaScript frameworks.

- **Key Technologies:** React.js, JavaScript (ES6+), HTML5, and CSS3.
- **Basic knowledge of Git version control system and familiarity with agile methodologies.**

- **Core Responsibilities :**

- Assist in the development and implementation of user interface components using React.js.
- Collaborate with the design team to translate UI/UX designs into functional web applications.
- Participate in code reviews and contribute to team best practices for coding standards.
- Test, debug, and optimize applications for maximum speed and scalability.
- Support the QA team in troubleshooting issues and performing user acceptance testing.

- Access: <https://app.recruitera.ai/job/frontend-development-intern--react-js--icura-cairo-egypt-3900?ref=DLJ24Vsf&igbTracker=902141851>

2. Backend Intern

Source: <https://www.linkedin.com/jobs/view/4399634843/>

This position is centered on server-side logic, database management, and ensuring high performance and responsiveness to requests from the frontend.

- **Focus Areas:** API development, server-side languages (Node.js), and database schema design.
- **Key Responsibilities:**
 - Develop and maintain server-side applications using Node.js.
 - Implement RESTful APIs and microservices architecture to facilitate communication between front-end systems and databases.
 - Collaborate with front-end engineers to integrate user-facing elements with server-side logic.
 - Ensure code quality and efficiency by performing code reviews and writing unit tests.
 - Work closely with stakeholders to gather requirements and translate them into technical specifications.
 - Monitor application performance and troubleshoot issues, ensuring high availability and reliability.
 - Stay updated with industry trends and best practices in backend development within healthcare.

1. Mahara-Tech (Information Technology Institute - ITI)

Mahara-Tech serves as the official e-learning arm of the ITI. It is designed to provide scalable, high-quality education to a broad audience.

- **Primary Focus:** Self-paced, online learning across diverse tracks including Web Development, Data Science, and Soft Skills.
- **Value Proposition:** Courses are free and widely recognized by Egyptian employers. They serve as an excellent entry point for those looking to build a foundation before applying for intensive 3-month or 9-month ITI diplomas.
- **Access:** maharatech.gov.eg

2. ITI Specialized Cybersecurity (Regional)

The ITI provides specialized, branch-specific tracks to cater to the growing demand for information security professionals in specific governorates.

- **Program Nature:** Intensive technical training focused on network security, penetration testing, and digital forensics.
- **Regional Focus (Giza):** Specific application forms are often released for regional branches to manage local intake.
- **Access:** Giza Application Portal

3. NTI "HireReady" & "Career Launch"

The **National Telecommunication Institute (NTI)** offers the "HireReady" and "Career Launch" initiatives, which are highly regarded for their practical, job-oriented curriculum.

- **Duration:** 6-month intensive training programs.
- **Core Tracks: Cybersecurity:** Defending infrastructure and managing risks.
- **Access:** NTI HireReady

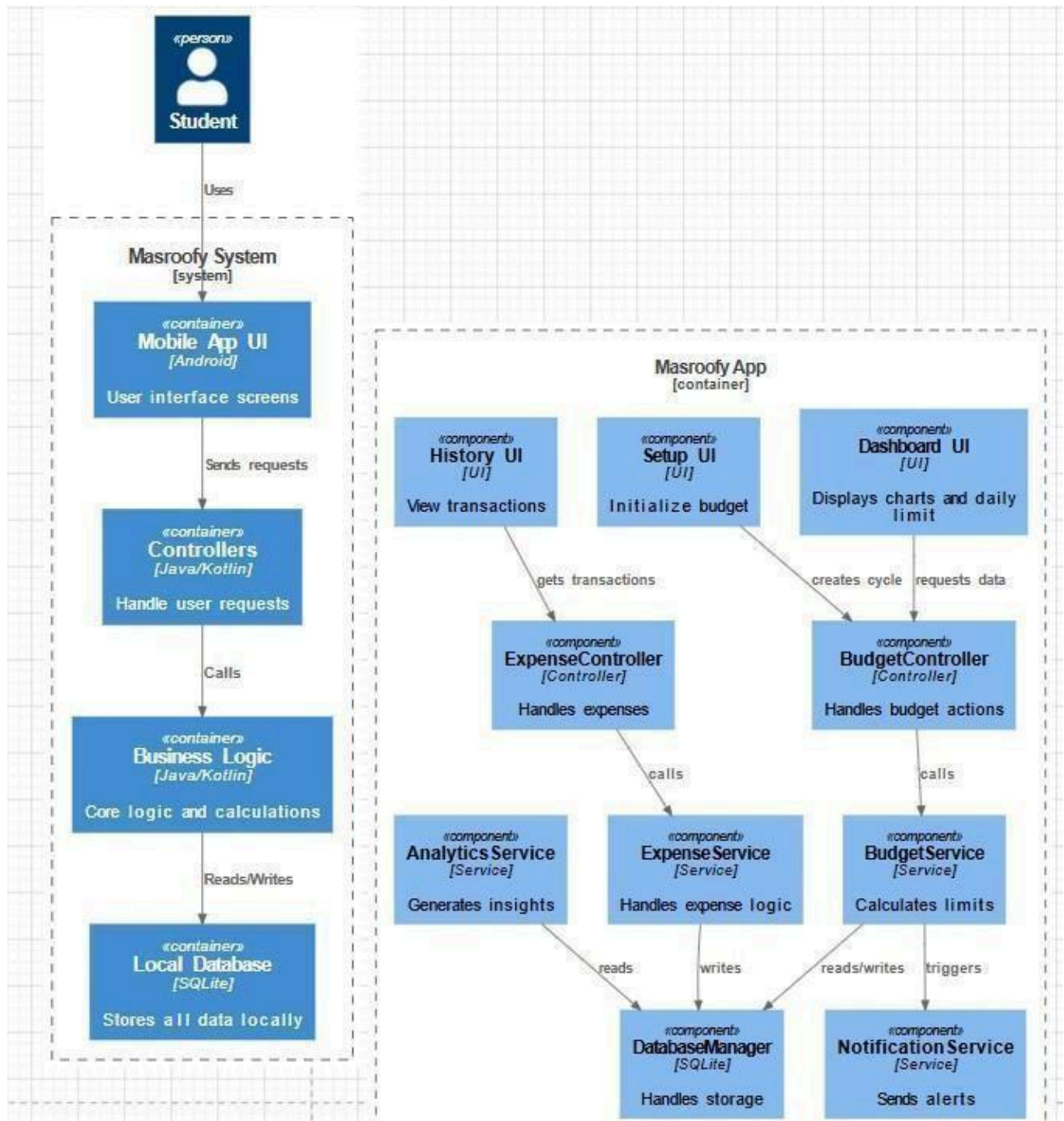
4. Manara AWS Training

This is a specialized collaboration focusing on **Amazon Web Services (AWS)** to bolster Egypt's presence in the global cloud computing market.

- **Primary Focus:** Cloud architecture, security, and infrastructure management using the AWS ecosystem.
- **Target Audience:** Software engineers and network beginners looking to migrate or build

System Models

I. Architecture Diagram





CS251: <Team Name> Project: <Masroofy>

Software Design Specification

Chosen Architecture: Layered Clean Architecture

In this approach, the system is organized into concentric layers where dependencies only point inward. This ensures that the core logic (the "why" of the app) remains independent of the database or the specific UI framework used.

Why this is suitable:

- **Separation of Concerns:** By decoupling the **UI**, **Controllers**, **Services**, and **Data**, you can modify the database (e.g., moving from SQLite to a Cloud DB) without touching the logic that calculates budget limits.
- **Testability:** You can test the BudgetService logic independently of the Android UI or the SQLite database.
- **Maintainability:** As a student project or a growing app, adding a new feature (like "Recurring Payments") only requires adding a new service and controller, rather than rewriting the entire app.

System Components & Packages

The system should be divided into the following four packages:

1. (View) Package:

- 0 **Components:** HistoryUI, SetupUI, DashboardU.
- o **Responsibility:** Only handles displaying data to the user and capturing input. It knows nothing about SQL or complex math.

2. (Controller) Package:

- 0 **Components:** ExpenseController, BudgetController.
- o **Responsibility:** Orchestrates the flow of data. It receives input from the View and tells the Services what to do.

3. (Service) Package:



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

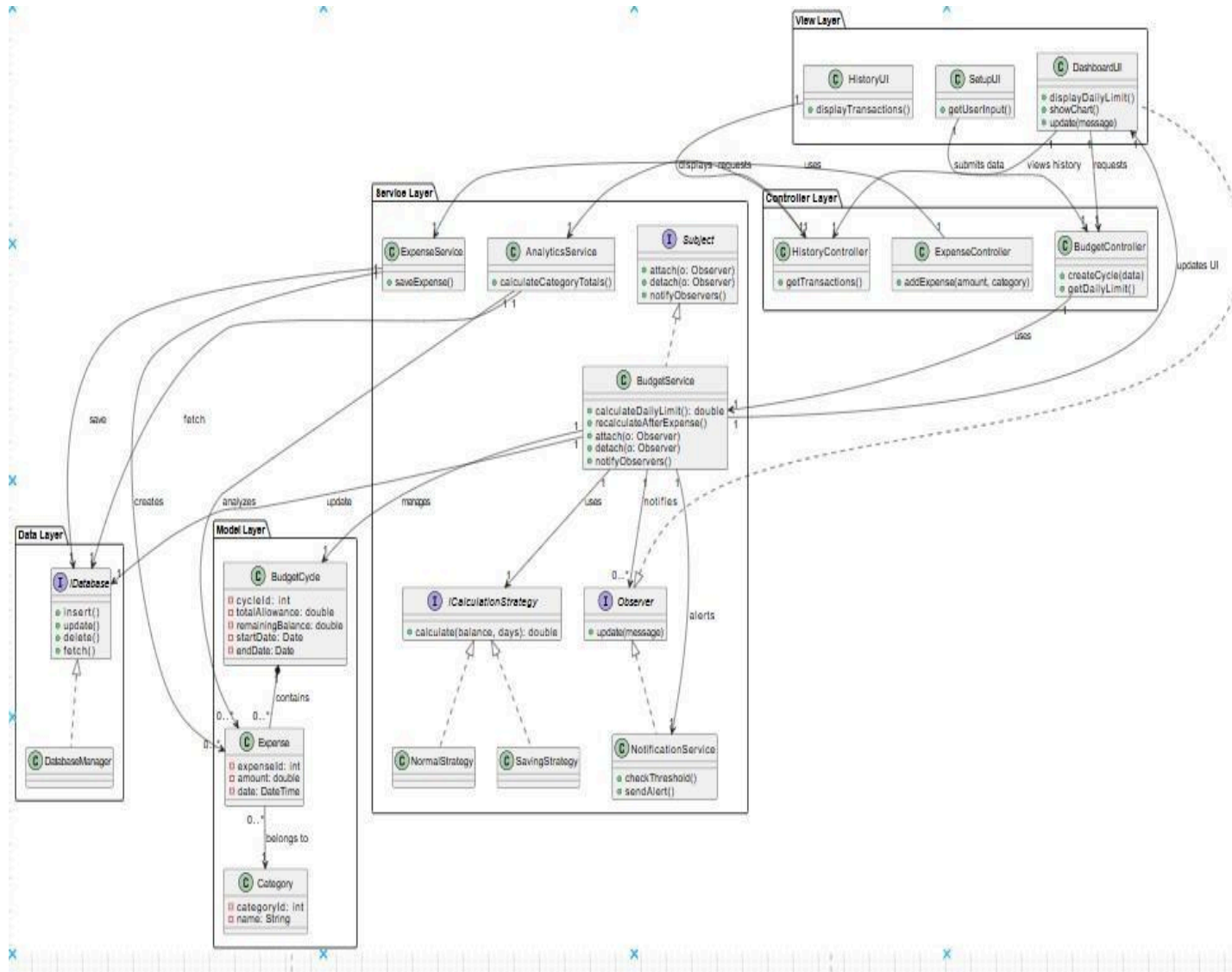
- 0 **Components:** ExpenseService, BudgetService, AnalyticsService.
 - o **Responsibility:** The "Brain" of the system. It contains the business rules, such as "How much daily budget is left?" or "Is this expense over the limit?"
4. **(Data/External) Package:**
- 0 **Components:** DatabaseManager (SQLite), NotificationService.
 - o **Responsibility:** Handles technical details like saving to a file or sending a push notification to the Android OS.

II. Class Diagram(s)



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

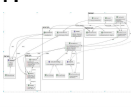


III. Class Descriptions



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

Class ID	Class Name	Description & Responsibility
1. 	DashboardUI	Responsible for displaying the user interface including daily budget limit and charts. Handles only presentation logic.
2.	SetupUI	Responsible for collecting initial budget input from the user during system setup.
3.	HistoryUI	Responsible for displaying previous transactions, expense history, and user spending records.
4.	BudgetController	Acts as mediator between View and BudgetService. Handles cycle creation and daily limits request
5.	ExpenseController	Responsible for handling user actions related to adding expenses and forwarding requests to expense service.
6.	HistoryController	Responsible for processing history and analytics requests, then forwarding them to AnalyticsService.
7.	BudgetService	Responsible for business logic related to budget calculation, remaining balance updates, daily limit recalculation, and observer notifications.
8.	ExpenseService	Responsible for recording, saving, and managing expense transactions.
9.	AnalyticsService	Responsible for generating analytical data such as category totals, summaries, charts, and spending reports.
10.	NotificationService	Responsible for checking threshold limits and sending alerts when spending exceeds limits or balance becomes low.
11.	Observer	interface for classes that receive automatic updates when the budget state changes.



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

12.	Subject	interface for managing observers through attach, detach, and notify methods.
-----	---------	--

Class ID	Class Name	Description & Responsibility
13.	IDatabase	interface defining database operations such as insert, update, delete, and fetch to ensure abstraction.
14.	DatabaseManager	Concrete implementation of IDatabase responsible for handling actual database communication.
15.	ICalculationStrategy	Strategy interface for flexible budget calculation logic
16.	SavingStrategy	Implements calculation logic optimized for saving mode to reduce daily spending.
17.	NormalStrategy	Implements standard calculation logic for regular daily budget distribution.
18.	BudgetCycle	Represents a budgeting period and stores total allowance, remaining balance, start date, and end date.
19.	Expense	represents a single expense transaction including amount, date, and related category.
20.	Category	Represents expense classification such as Food, Transport, Bills, or Entertainment.

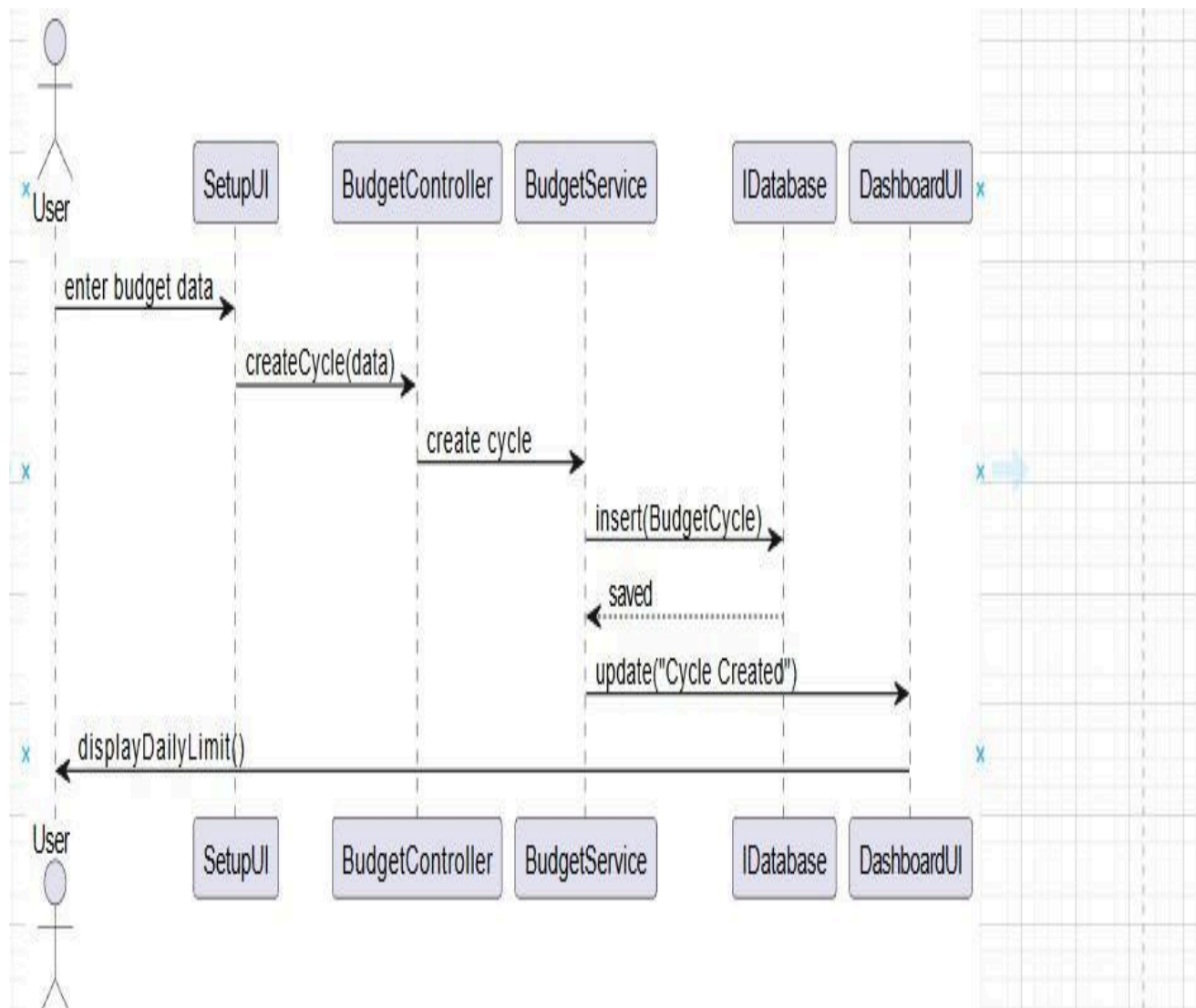
IV. Sequence diagrams

User Story 1 :



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

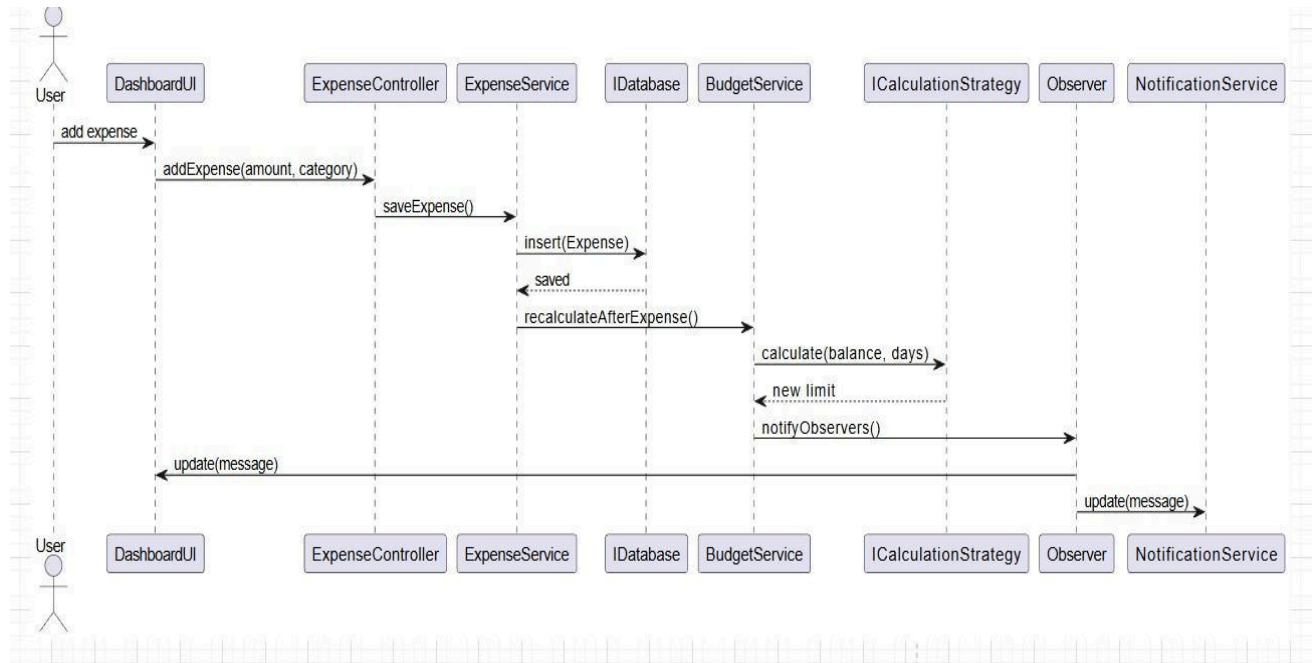


User Story 2 :



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

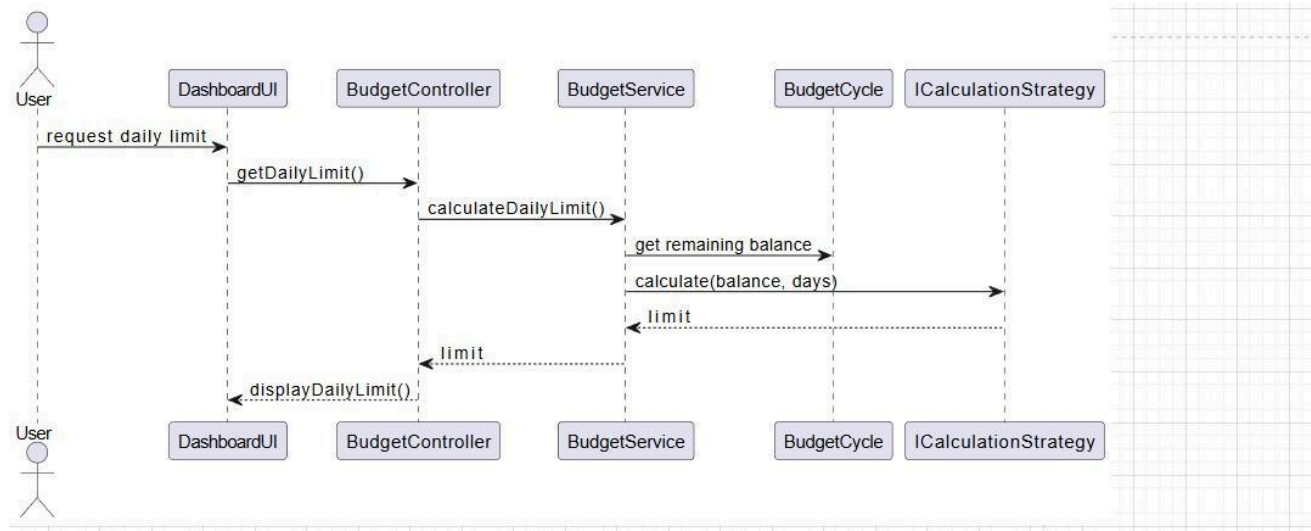


User Story 3 :



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

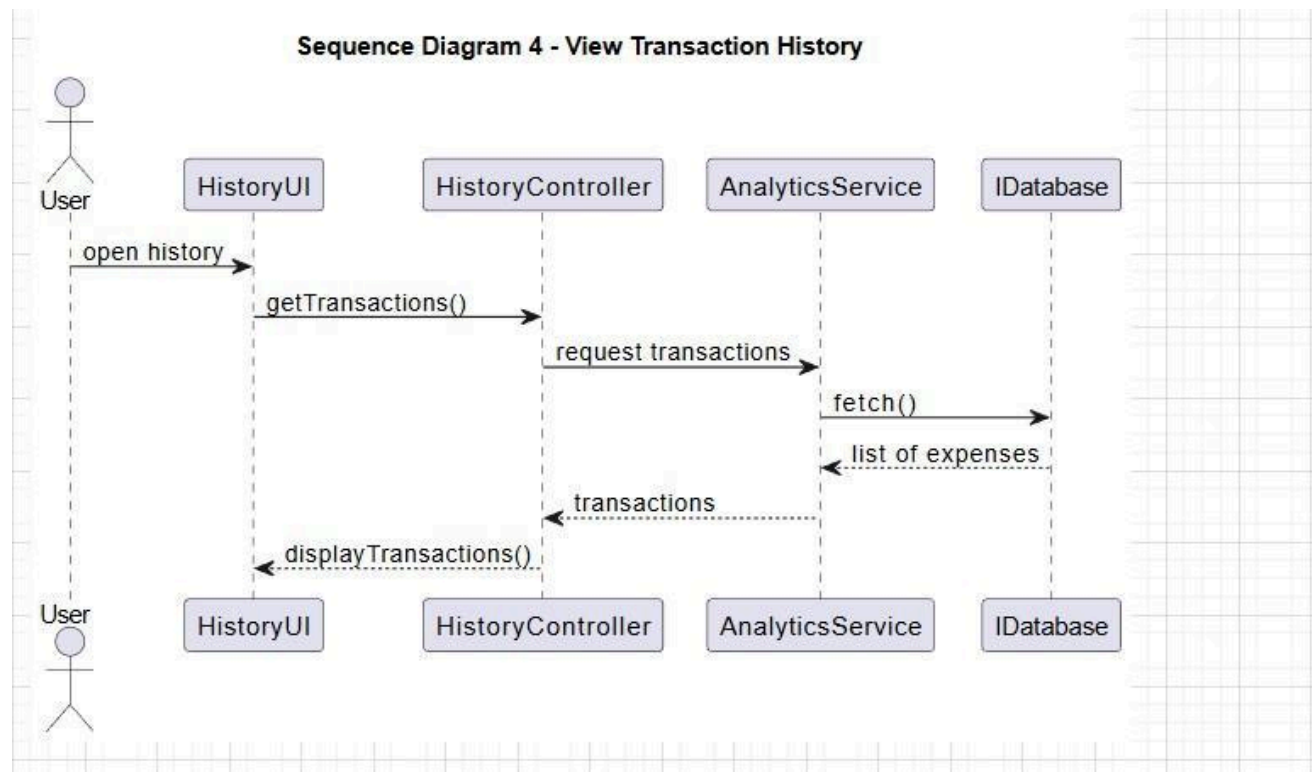


User Story 4 :



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

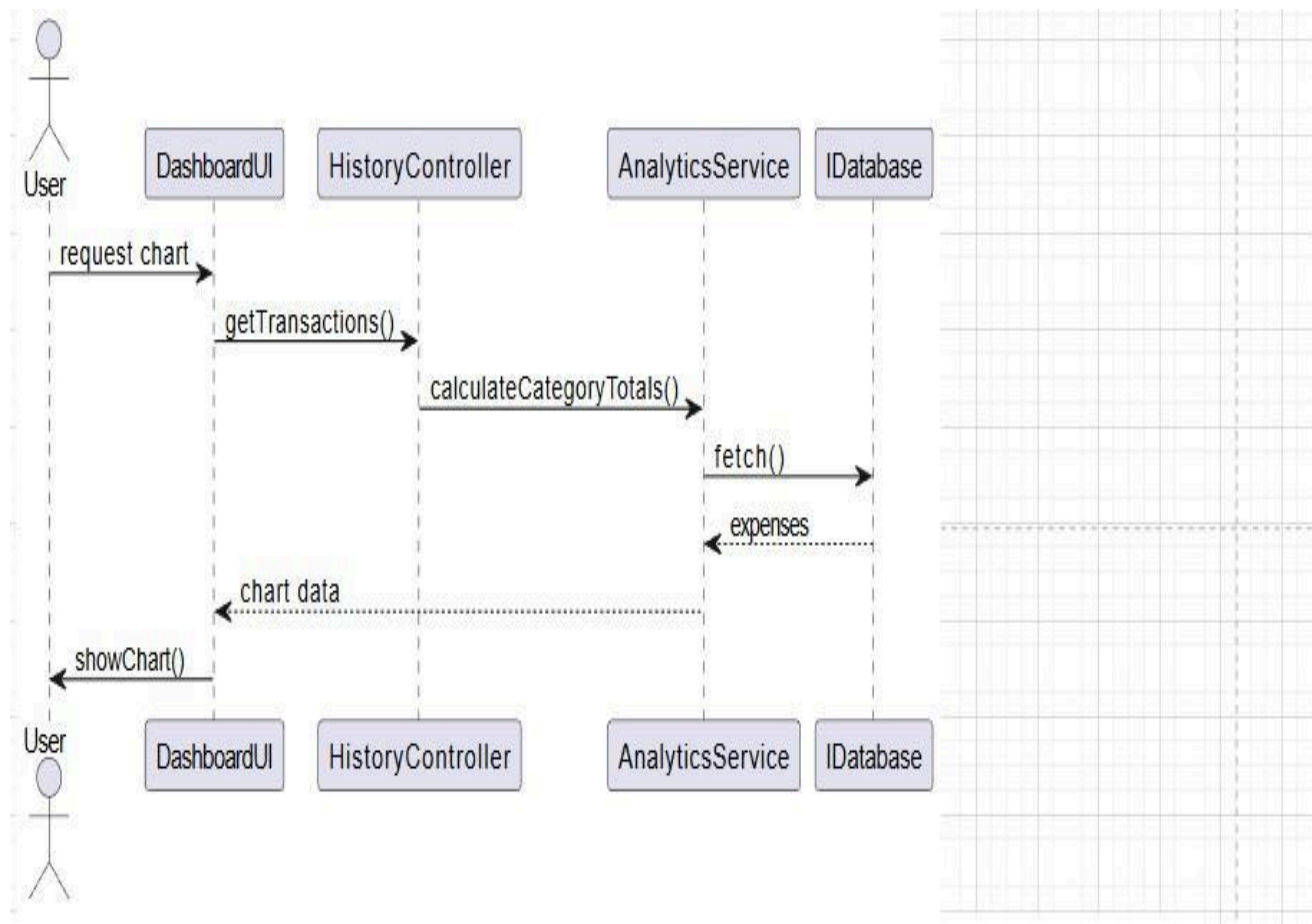


User Story 5 :



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

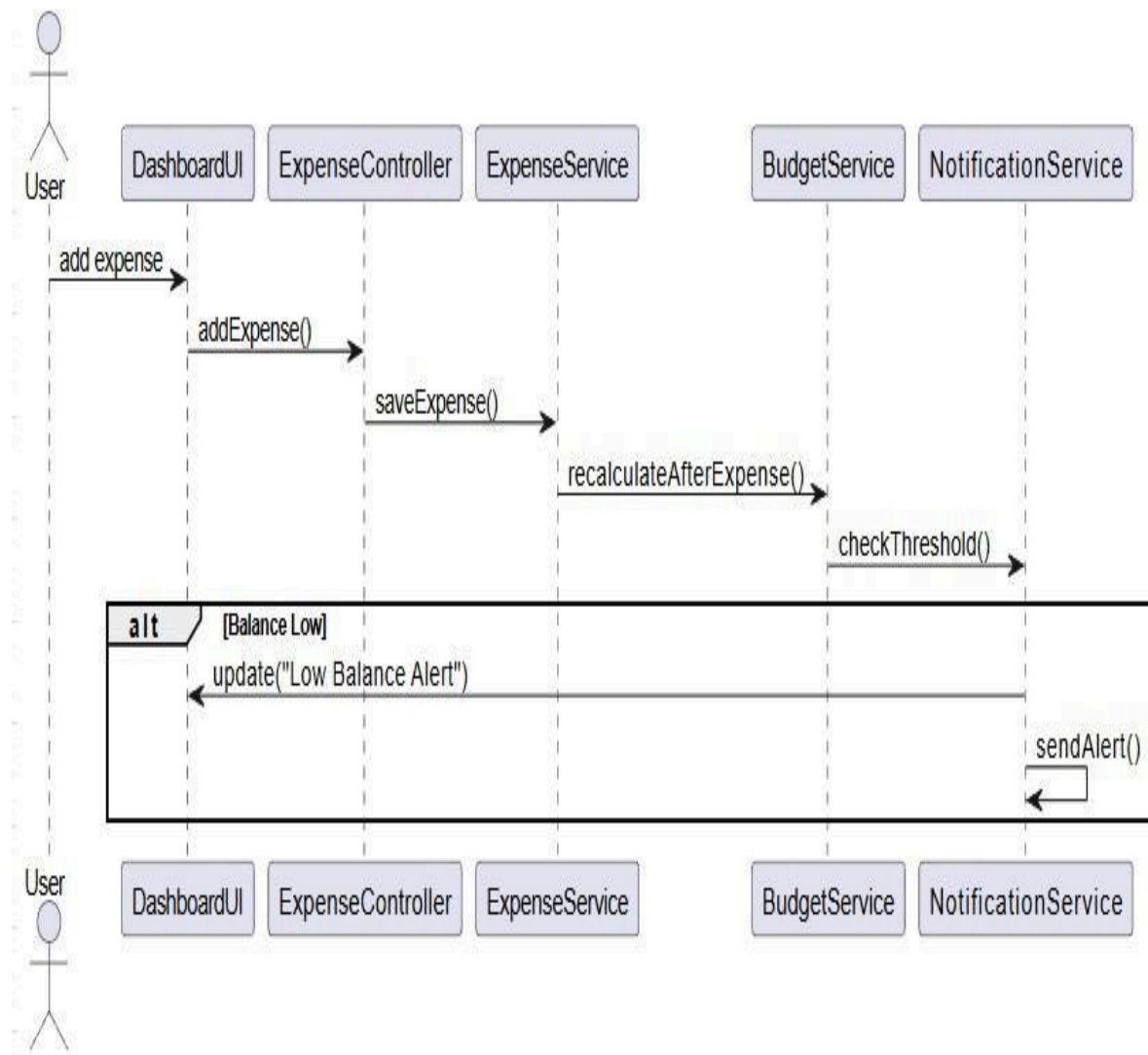


User Story 6 :



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

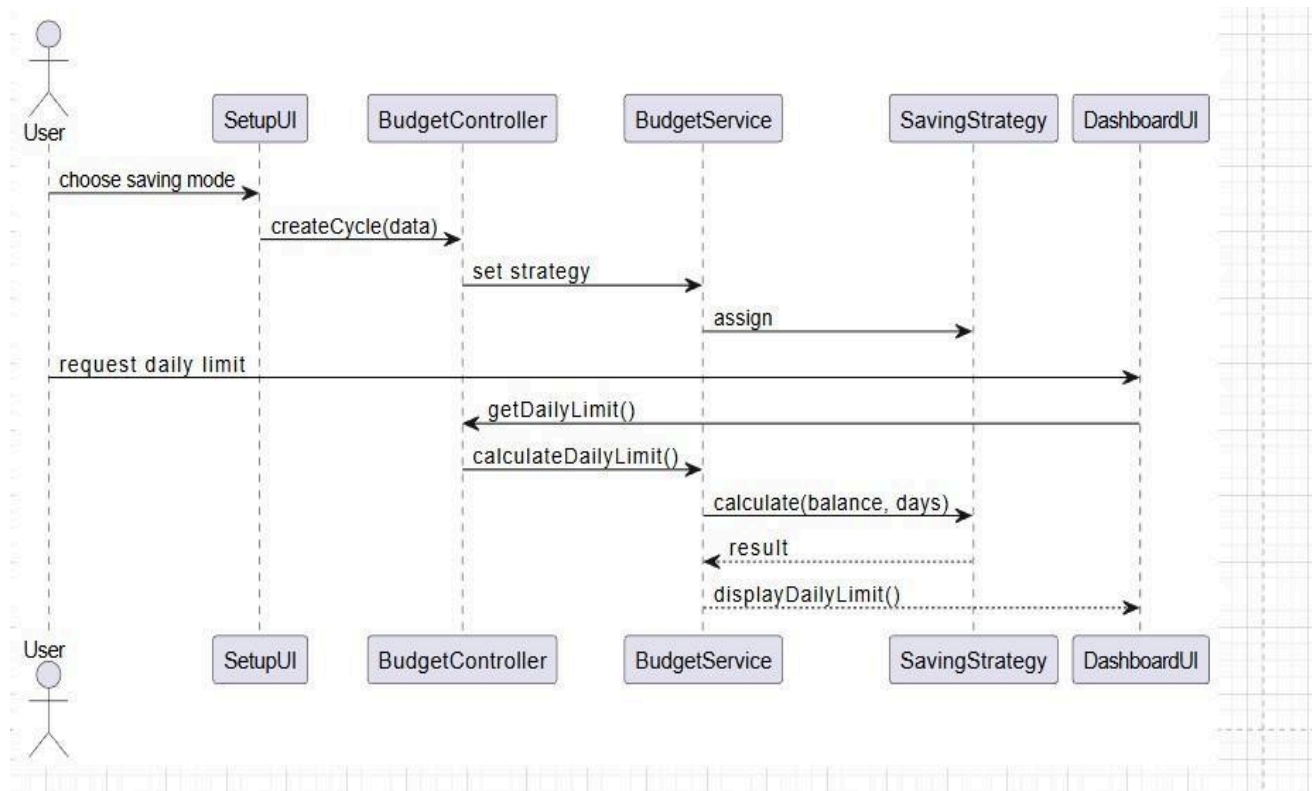




CS251: <Team Name> Project: <Masroofy>

Software Design Specification

User Story 7 :



Class - Sequence Usage Table

CU – FCAI – CS251 Introduction to Software Engineering – 2026 - Software Design Specifications
Prepared by Mostafa Saad and Mohammad El-Ramly V1.0

Edited by Mohamed Samir, Updated to V2.0 by Mohammad El-Ramly 10/4/2020 and V3.0 25/5/2021, 20/4/2025



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

Sequence Diagram	Class Used	All Methods Used
User Story 1- Setup Budget cycle	SetupUI, BudgetController, BudgetService, BudgetCycle, IDatabase, DashboardUI	+getUserInput(): data +createCycle(data): void +insert(): Boolean +calculateDailyLimit(): double +displayDailyLimit(): void +update(message): void
User Story 2-Add Expense	DashboardUI, ExpenseController, ExpenseService, Expense, IDatabase, BudgetService, ICalculationStrategy, NotificationService	+addExpense(amount, category): void +saveExpense(): void +insert(): Boolean +recalculateAfterExpense(): void +calculate(balance, days): double +checkThreshold(): void +sendAlert(): void +update(message): void
User Story 3- View Daily Limit	DashboardUI, BudgetController, BudgetService, BudgetCycle, ICalculationStrategy	+getDailyLimit(): double +calculateDailyLimit(): double +calculate(balance, days): double +displayDailyLimit(): void
User Story 4-View Transaction History	HistoryUI, HistoryController, AnalyticsService, Expense, IDatabase	+displayTransactions(): void +getTransactions(): List<Expense> +fetch(): List +calculateCategoryTotals(): Map
User Story 5- View Analytics Chart	DashboardUI, HistoryController, AnalyticsService, Expense, Category, IDatabase	+showChart(): void +getTransactions(): List<Expense> +fetch(): List +calculateCategoryTotals(): Map +getName(): String

CU – FCAI – CS251 Introduction to Software Engineering – 2026 - Software Design Specifications

Prepared by Mostafa Saad and Mohammad El-Ramly V1.0

Edited by Mohamed Samir, Updated to V2.0 by Mohammad El-Ramly 10/4/2020 and V3.0 25/5/2021, 20/4/2025

| 31



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

User Story 6- Low Balance Alert	DashboardUI, ExpenseController, ExpenseService, BudgetService, NotificationService, Observer	+addExpense(amount, category): void +saveExpense(): void +recalculateAfterExpense(): void +checkThreshold(): void +sendAlert(): void +notifyObservers(): void +update(message): void
User Story 7-Change Calculation +update(message): void Strategy	SetupUI, BudgetController, BudgetService, SavingStrategy, NormalStrategy, DashboardUI	+getUserInput(): data +createCycle(data): void +calculateDailyLimit(): double +calculate(balance, days): double +displayDailyLimit(): void



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

V. State Diagram

- **Transaction State Diagram Description:**
- **The Transaction object goes through several states during its lifecycle in the Budget Management System.**
- **Initially, a transaction is created in the Created state when the user enters the transaction details. Once the user submits the transaction, it transitions to the Pending state, where it awaits approval.**
- **From the Pending state, the transaction can either be Approved or Rejected based on the validation or user decision. If the transaction is approved, it moves to the Approved state. After processing, the transaction reaches the Completed state, indicating that it has been successfully executed and recorded in the system.**



CS251: <Team Name> Project: <Masroofy>

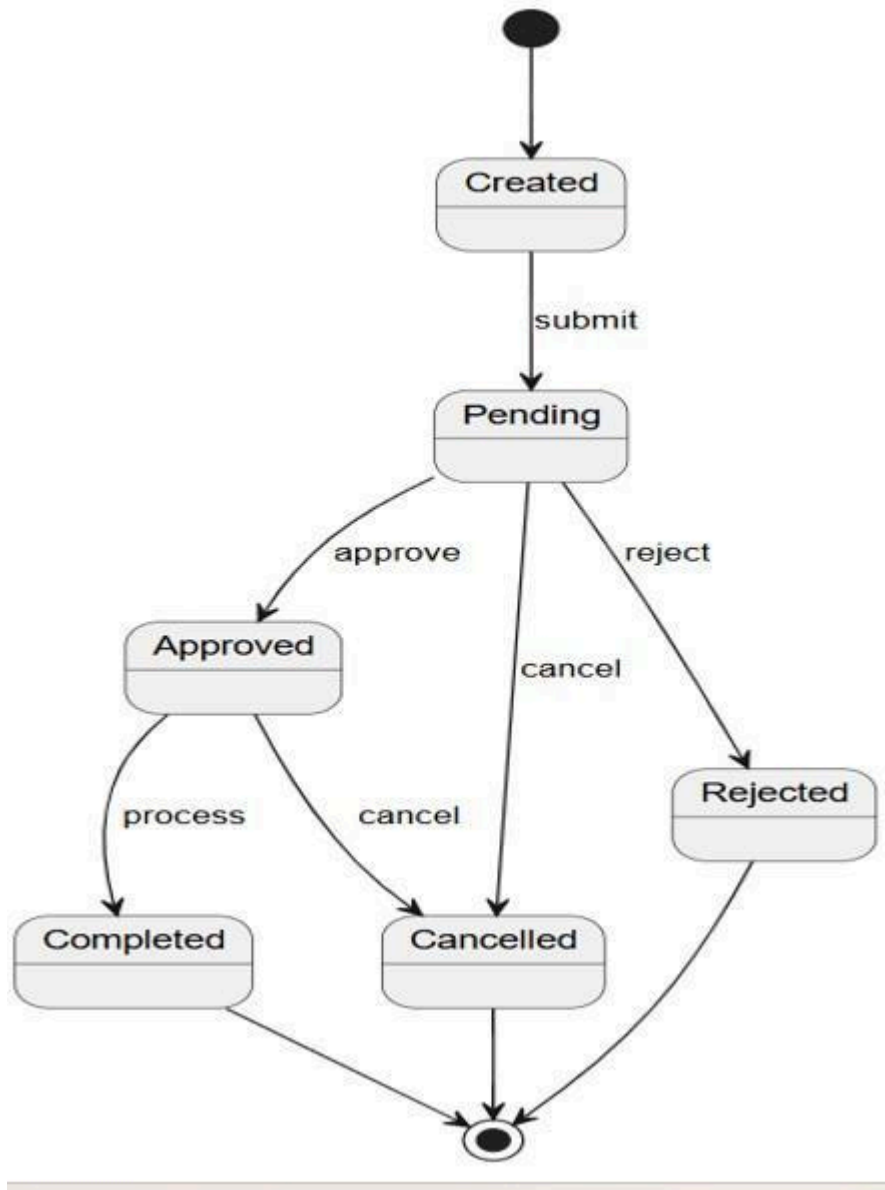
Software Design Specification

- At any point before completion, the transaction can be Cancelled either from the Pending or Approved state. Once a transaction is Rejected, Cancelled, or Completed, it reaches a final state and no further transitions are allowed.



CS251: <Team Name> Project: <Masroofy>

Software Design Specification



VI. SOLID Principles



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

- Single Responsibility Principle (SRP)

Each class in the system is responsible for one specific functionality only:

- ExpenseService handles expense operations such as adding and saving expenses.
- BudgetService is responsible for calculating daily limits and updating balances.
- AnalyticsService generates reports, statistics, and category summaries.
- NotificationService checks thresholds and sends alerts.
- DashboardUI is responsible only for displaying data to the user.

This separation improves readability and makes the system easier to maintain, test, and update.

- Open/Closed Principle (OCP)

The system is open for extension but closed for modification.

The ICalculationStrategy interface allows adding new calculation methods without changing existing code.

Examples:

- SavingStrategy
- NormalStrategy

Future strategies such as:

- StrictSavingStrategy • StudentStrategy can be added easily by implementing the same interface.
-

- Liskov Substitution Principle (LSP)

Any class implementing ICalculationStrategy can replace another strategy without affecting the system behavior.



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

Examples:

- SavingStrategy
- NormalStrategy

Both can be used inside BudgetService through the same interface.

This ensures flexibility and correctness.

- Interface Segregation Principle (ISP)

Interfaces are small, focused, and specific.

- ICalculationStrategy contains only calculation-related behavior.
- IDatabase contains only database operations such as insert, update, delete, and fetch.

Classes depend only on methods they actually need.

-Dependency Inversion Principle (DIP)

High-level modules depend on abstractions rather than concrete implementations.

Examples:

- BudgetService depends on ICalculationStrategy instead of specific strategies.
- ExpenseService, BudgetService, and AnalyticsService depend on IDatabase instead of DatabaseManager directly.

This reduces coupling and improves maintainability.

VII. Design Patterns

CU – FCAI – CS251 Introduction to Software Engineering – 2026 - Software Design Specifications
Prepared by Mostafa Saad and Mohammad El-Ramly V1.0

Edited by Mohamed Samir, Updated to V2.0 by Mohammad El-Ramly 10/4/2020 and V3.0 25/5/2021, 20/4/2025

| 37



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

Observer Pattern

A. Subject: BudgetService

The BudgetService acts as the subject being observed.

Whenever the budget changes after adding expenses or recalculating the daily limit, it notifies all registered observers.

B. Observers

The following classes act as observers:

- DashboardUI updates displayed data and charts.
- NotificationService sends alerts when balance becomes low.

Benefit

- Automatic updates when budget state changes.
- Loose coupling between components.
- Easy to add new observers later.

Strategy Pattern

Implemented using the ICalculationStrategy interface.

Different strategies such as:

- SavingStrategy
- NormalStrategy

define different calculation behaviors.



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

The system can switch strategies dynamically depending on user mode or preferences.

Benefit

- Eliminates complex conditional logic.
 - Makes the system easy to extend.
 - Supports runtime behavior changes.
-

- MVC Architecture

The system follows the Model-View-Controller (MVC) architectural pattern to ensure clear separation of concerns and improve maintainability.

1. Model

The Model layer represents the core data and business entities of the system.

It includes classes such as:

- BudgetCycle
- Expense
- Category

These classes store application data and define system structure.



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

2. View

The View layer is responsible for presenting data to the user and handling interface interactions.

It includes:

- DashboardUI
- HistoryUI
- SetupUI

These classes display daily budget limits, reports, charts, and collect user input.

3. Controller

The Controller layer acts as an intermediary between the View and business logic.

It processes user requests and delegates them to the appropriate services.

Examples:

- BudgetController
 - ExpenseController
 - HistoryController
-

4. Service Layer (Business Logic)

Although not part of traditional MVC, a Service Layer is added to manage business logic.

Classes include:



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

- BudgetService
- ExpenseService
- AnalyticsService
- NotificationService

These classes perform calculations, manage transactions, and interact with the database.

- System Flow

The system interaction follows this sequence:

User → View → Controller → Service → Model → Database

Example:

1. User enters an expense.
 2. View sends request to ExpenseController.
 3. Controller calls ExpenseService.
 4. ExpenseService stores data in database.
 5. BudgetService recalculates balance.
 6. Observers are notified.
 7. Dashboard updates automatically.
-

- Benefits of the Design

- Clear separation between UI, logic, and data.
- Improved maintainability.



CS251: <Team Name> Project: <Masroofy>

Software Design Specification

- Easy scalability for future features.
- Reusable and testable components.
- Reduced coupling using interfaces and patterns.
- Better code organization and readability.

Tools

We used planet uml and draw.io

Ownership Report

Item	Owners
Mohamed yasser hamed	State &sequence diagram
Mohamed Yasser	Class diagram& architecture design
Omar abdelaziz	<i>Class diagram</i>
Mostafa Ayman	<i>Sequence diagram</i>